

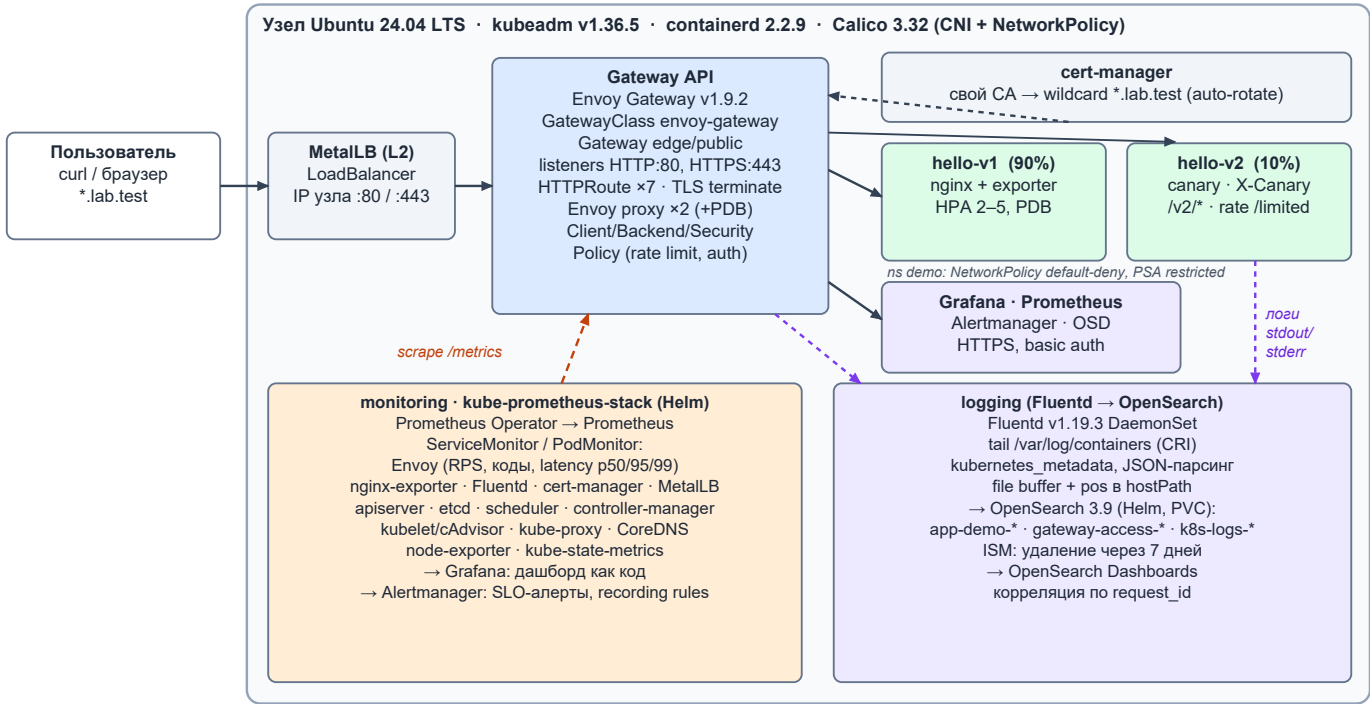
Паспорт решения: kube-gateway-lab

Репозиторий: <https://github.com/SharPixX/hack> (ветка main). Развёртывание одной командой `sudo ./deploy.sh` на чистой Ubuntu 24.04; в конце автоматически выполняется сквозной `smoke-test` (~40 проверок).

1. Архитектура и состав решения

Параметр	Значение
Версия Kubernetes	v1.36.5 (kubeadm/kubelet/kubectl из pkgs.k8s.io, пакеты на hold). 1.36 — новейшая minor-версия, поддерживаемая всем стеком (Envoy Gateway 1.9, Calico 3.32)
Способ развёртывания Kubernetes	kubeadm (конфиг v1beta4 из шаблона), containerd 2.2.9 + runc 1.4.3 + CNI plugins 1.9.1 из официальных релизов с проверкой sha256, CNI Calico (VXLAN). По умолчанию single-node, multi-node — через Ansible inventory (kubeadm join)
Реализация Gateway API	Envoy Gateway v1.9.2 (Gateway API v1.6.1, Envoy 1.39); LoadBalancer — MetalLB 0.16 (L2); TLS — cert-manager 1.21
Инструменты автоматизации	Ansible (ansible-core 2.21, 7 полей) + Helm 3.22 (через kubernetes.core.helm) + Kustomize ; обёртки deploy.sh / Makefile; GitHub Actions (lint + e2e)
Инструмент логирования	Fluentd v1.19.3 (DaemonSet) → OpenSearch 3.9 → OpenSearch Dashboards
Способ развёртывания Prometheus	Helm-чарт kube-prometheus-stack 91.9.0 (Prometheus Operator v0.94): Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics; таргеты — ServiceMonitor/PodMonitor
ОС, на которой тестировалось	Ubuntu 24.04 LTS : e2e-деплой на чистом раннере GitHub Actions ubuntu-24.04 (x86_64, 4 vCPU/16 GB) на каждый push — двойной прогон + smoke-test

Архитектурная схема: пользователь → Gateway API → приложение, мониторинг и логирование



Развёртывание: `sudo ./deploy.sh` → Ansible (поли) → Helm + Kustomize · CI: GitHub Actions — lint + e2e на ubuntu-24.04

2. Реализованный функционал — обязательные требования

Требование	Как реализовано технически	Обоснование выбора	Как проверить
1. Kubernetes	Ansible: node_prep (swap off, модули ядра, sysctl) → kube_packages → containerd → kubeadm init. Метрики etcd/scheduler/controller-manager/kube-proxy открыты для Prometheus в kubeadm-конфиге	kubeadm — приоритет T3, вендор-нейтрально; upstream-бинарники с sha256 — воспроизводимо и без Docker	kubectl get nodes -o wide, kubectl version
2. Веб-приложение	nginx 1.30.5 (unprivileged) в 2 версиях: v1 (HPA 2–5) и v2 (canary). GET / → Hello World!; JSON access-лог → stdout, error-лог → stderr; sidecar nginx-prometheus-exporter	Простой OSS-сервер, non-root образ; структурированные логи сразу пригодны для поиска	curl http://hello.lab.test/
3. Gateway API	GatewayClass envoy-gateway (+EnvoyProxy) → Gateway edge/public (HTTP:80, HTTPS:443, allowedRoutes по метке namespace) → 7 HTTPRoute → Service hello-v1/v2. Адрес Gateway выдаёт MetalLB	Envoy Gateway — CNCF-проект, полная конформность Gateway API, расширения через policy attachment, метрики/логи Envoy	kubectl get gateway -A (PROGRAMMED=True), curl по §5 README
4. Мониторинг	kube-prometheus-stack; ServiceMonitor/PodMonitor: Envoy, nginx, Fluentd, Envoy Gateway, cert-manager, MetalLB, control plane; дашборд Grafana и PrometheusRule (SLO-алерты + recording rules)	Де-факто стандарт; таргеты описаны декларативно рядом с приложением	Prometheus: sum by (job)(up); Grafana → дашборд kube-gateway-lab
5. Логирование	Fluentd DaemonSet: tail CRI-логов, kubernetes_metadata, JSON-парсинг nginx/Envoy, file buffer → OpenSearch (индексы app-demo, gateway-access, k8s-logs; шаблоны, ISM 7d, index patterns создаются автоматически)	Fluentd — CNCF graduated, официальный образ с плагином OpenSearch; OpenSearch (Apache 2.0) — полнотекстовый поиск	запрос /?probe=X → поиск X в app-demo-* (команды в §7 README)
6. Ubuntu 24.04	Проверка ОС в deploy.sh и playbook; e2e в CI на чистом раннере ubuntu-24.04	Доказуемая воспроизводимость на целевой ОС при каждом изменении	вкладка Actions → job «E2E - kubeadm cluster on Ubuntu 24.04»
7. Автоматизация	sudo ./deploy.sh: pre-flight → venv с закреплённым Ansible → site.yml → smoke-test. Идемпотентно: kubeadm только при отсутствии admin.conf, Helm-модуль сравнивает values, kubectl apply	Одна команда без ручных шагов; Ansible + Helm + Kustomize — стандартный стек конфигурации	повторный sudo ./deploy.sh; в CI деплой запускается дважды
8. Документация	README: архитектура (mermaid), версии, требования, установка, проверки, ограничения; этот паспорт	Эксперт может пройти путь без автора	README.md в корне

Дополнительные улучшения

Что реализовано	Как технически	Зачем	Как проверить
TLS + собственный PKI	cert-manager: self-signed root → CA ClusterIssuer → wildcard *.lab.test, ротация за 15 дн.; ClientTrafficPolicy TLS ≥ 1.2	HTTPS без ручной работы с ключами	sudo make ca; curl --cacert lab-ca.crt https://hello.lab.test/
Traffic splitting, маршруты по заголовку/пути/hostname	weighted backendRefs 90/10; X-Canary; /v1, /v2 + URLRewrite; ResponseHeaderModifier; 5 hostname	Канареечные релизы средствами Gateway API	цикл из 200 запросов к /info (§5)
Редирект HTTP→HTTPS, UI через Gateway	RequestRedirect 301; HTTPRoute в monitoring/logging (cross-namespace)	Единая точка входа	curl -I http://grafana.lab.test → 301
Rate limit, retry, circuit breaker	BackendTrafficPolicy: 5 rps на /limited, 2 retry с backoff, лимиты соединений	Защита backend от перегрузки	20 быстрых запросов к /limited → 429
Basic auth на Gateway	SecurityPolicy + httpasswd-Secret, пароль генерируется при деплое	UI без своей авторизации закрыты	401 без пароля, 200 с паролем
HTTP RED-метрики, дашборд и алерты как код	Envoy upstream_rq_xx/time, nginx, CPU/RAM; tools/gen-dashboard.py; 8 своих алертов + recording rules	Наблюдаемость по SLO	Grafana, Prometheus → Alerts
Поиск логов и корреляция	OpenSearch + Dashboards, request_id в логгах Envoy и nginx; ISM retention	Расследование инцидентов	OSD Discover: uri:*probe*
Безопасность	PSA restricted, NetworkPolicy default-deny, non-root/RO FS, без секретов в Git, sha256-pinning, gitleaks	Минимальные привилегии	kubectl -n demo get netpol
Надёжность	Envoy ×2 + PDB, HPA + PDB, maxUnavailable 0, preStop, probes, буферы Fluentd на диске	Обновления без простоя	kubectl -n demo get hpa, pdb
CI/CD	lint: yamllint, ansible-lint (production), shellcheck, helm template, kubeconform -strict по схемам из CRD закреплённых чартов, gitleaks, trivy; e2e: kubeadm на ubuntu-24.04, повторный деплой, smoke-test	Ошибки ловятся до деплоя; доказана работа на целевой ОС	вкладка Actions, Job Summary

3. Ревью работы и потенциальное масштабирование

Главная особенность. Решение не только разворачивается одной командой с нуля (kubeadm, а не kind), но и само доказывает свою работоспособность: smoke-test проверяет каждое требование — от статусов Gateway/HTTPRoute и sanity 90/10 до появления конкретного HTTP-запроса в OpenSearch и метрик в Prometheus. Это же автоматически выполняется в CI на чистой Ubuntu 24.04 с повторным деплоем для проверки идемпотентности, а манифесты строго валидируются по схемам CRD именно тех версий, что устанавливаются.

Самое сложное решение. Как опубликовать Gateway на bare-metal kubeadm без облачного балансировщика: NodePort (нестандартные порты, нет адреса в status Gateway), hostNetwork для Envoy (обход модели Gateway API) или MetalLB. Выбран MetalLB L2 с IP узла по умолчанию: Gateway получает настоящий адрес в status.addresses и стандартные 80/443, а для multi-node пул меняется одной переменной. Аналогично для логов OpenSearch выбран вместо Loki: он работает с официальным образом Fluentd (для Loki нужен сторонний плагин и своя сборка образа) и даёт полнотекстовый поиск.

Предложения по развитию (и что для этого нужно):

Направление	Что добавить	Что потребуется
HA control plane	3 control-plane узла, kube-vip/keepalived VIP, stacked или внешний etcd	3+ ВМ, свободный VIP в сети
GitOps	Argo CD / Flux синхронизируют k8s/ и Helm values из Git, drift detection	Только ресурсы кластера
Телеком: протоколы 5G Core / IMS	GRPCRoute для HTTP/2-сервисов 5G SBA; TCPRoute/UDPRoute (SIP, Diameter) — experimental-канал Gateway API, уже поддержан Envoy Gateway	Тестовые NF (open5gs, Kamailio)
Телеком: SLO и надёжность	SLO 99.99% на Gateway с multi-window burn-rate алертами, синтетические пробы (blackbox-exporter), ретеншн метрик в Thanos/Mimir	S3-совместимое хранилище (MinIO/Ceph)
Телеком: data plane	Multus + SR-IOV/DPDK для пользовательского трафика, отдельные сети управления/данных	NIC с SR-IOV, CPU pinning, hugepages
Безопасность	OIDC вместо basic auth (SecurityPolicy oidc), mTLS между сервисами, OpenSearch security plugin, Kyverno/Gatekeeper, подпись образов cosign	IdP (Keycloak), PKI-политики
Трассировка	OpenTelemetry Collector + Tempo/Jaeger, трейсинг Envoy, exemplars в Grafana	Ресурсы кластера
Глобальный rate limit	Envoy Gateway global rate limit (ratelimit + Redis) для нескольких реплик	Redis (в кластере или managed)
Air-gapped установка	Зеркало образов и чартов (Harbor, OCI), офлайн-пакеты apt	Внутренний registry

Известные ограничения (подробно — README §11): single-node по умолчанию (control plane не HA, данные на local-path); MetalLB по умолчанию отдаёт IP узла (порты 80/443 должны быть свободны); security plugin OpenSearch отключён (API только внутри кластера, UI за basic auth); metrics-server с --kubelet-insecure-tls; локальный (не глобальный) rate limit.